Name: Caleb Sorell

Date: October 16, 2026

Assignment: CS 470 Final Reflection

Presentation Link: https://youtu.be/xOUsut8FsZw

Experiences and Strengths

I first learned how to replatform an existing web application using containerization. I did this by using Docker Engine, writing Docker files, building Docker Images, and creating multi-container environments with a single Docker Compose file. I have also learned how to package an application, so its components communicate on a shared network before moving to the cloud. Also, I gained hands-on experience migrating a MEAN stack into a serverless AWS environment, allowing me to host an Angular frontend in Amazon S3. With the proper set ups, I was able to route event-driven requests through Amazon API Gateway to scripts inside AWS Lambda. Finally, I learned the process of transitioning a document-based MongoDB database to a key-value NoSQL in Amazon DynamoDB.

My strong attention to detail and my ability to handle a complex workload would be my core strengths as a developer. By being able to take an existing full-stack codebase and transfer it into a cloud architecture. I believe staying detail-oriented is crucial in this field, because everything must line up perfectly for an application to work. Furthermore, I always make it a priority to lock down the security systems to protect the infrastructure of an application. These skills that I have learned prepare me to step into a Full-Stack Cloud Developer role, and into an IT Specialist, Cybersecurity, or Cloud Security Engineer focusing on securing cloud environments and managing access controls.

Planning for Growth

By using a serverless approach, it provides advantages for both management and scale. The application's backend is divided into separate AWS Lambda functions. This reason is so when an error or overload in one function happens, it will not crash the system. For example, if one function goes down, users can still browse others without any problems. Also, AWS will only scale specific functions that are receiving higher traffic.

When you analyze cost, paying for a fixed rate to run 24/7 servers is a predictable pay model. However, using a serverless pay-for-use policy is far more cost-efficient. The reason for this is whenever your servers hit low or there is no traffic you pay less for your servers. You can also set strict budget limits and alerts in AWS Budgets to notify the team if spending approaches threshold.

Here are some pros and cons of serverless architecture when planning it for future expansion:

- Pros: Going serverless removes nearly all maintenance. No longer do developers need to spend time installing OS updates, managing hardware, or manually maintaining server uptime and time. With all that handled through AWS, it allows developers more time to focus on development.
- Cons: Going full serverless model locks an application into Amazon's ecosystem, which in turn reduces individual independence control over servers. Also, open API Gateway is vulnerable to bots spamming requests, this could inflate the pay-per-use bill or throttle the server.

Overall, elasticity and the pay-for-service model are the deciding factors for future growth. Using the pay-for-service policy helps to protect a budget during quiet times. Then the elasticity guarantees when traffic exceeds capacity, AWS will scale in real time to prevent crashes and customer dissatisfaction.